

Guía de Práctica Guiada de Laboratorio

Tema 2.1: Herencia y Polimorfismo — Caso de Estudio «Vehículo, Auto y Motocicleta»

1. Objetivo y Fundamentos Técnicos (80 minutos)

Objetivo General: Que el estudiante implemente de manera práctica una jerarquía de clases compuesta por la clase base Vehículo y sus subclases derivadas Auto y Motocicleta en C++, aplicando los modificadores de acceso protected, la redefinición de métodos con virtual y override, la delegación explícita de constructores base, el mecanismo de enlace dinámico (vtable) y el diseño seguro con destructores virtuales.

Conceptos Clave del Tema 2.1:

- **Relación 'Es un' (Is-a):** A diferencia de la composición ("Tiene un"), la herencia representa especialización.
- **Modificador protected:** Oculta atributos y métodos al exterior (igual que private), pero permite su acceso directo a clases derivadas.
- **Polimorfismo Dinámico:** Permite tratar objetos de subclases (Auto, Motocicleta) a través de punteros a la clase base (Vehículo*).
- **Destructor Virtual:** Garantiza la liberación completa de recursos cuando se elimina un objeto derivado a través de un puntero base.

2. Código Fuente Base del Sistema

El siguiente programa implementa la jerarquía de transporte. Exhibe el uso de miembros protegidos, llamadas a métodos base mediante Vehículo::, funciones virtuales sobrescritas y un arreglo de punteros polimórficos.

Módulo 1: Clase Base Vehículo

```
#include <iostream>
#include <string>
#include <vector>

// =====
// CLASE BASE: Vehículo
// =====

class Vehículo {
protected:
    std::string marca;
    std::string modelo;
    double velocidadActual;
public:
    Vehículo(std::string m, std::string mod)
        : marca(m), modelo(mod), velocidadActual(0.0) {}

    // Destructor virtual imprescindible para evitar fugas de memoria
    virtual ~Vehículo() {
        std::cout << " [Destructor Vehículo] Liberando base para: " << marca << " " << modelo << std::endl;
    }

    // Método virtual para enlace dinámico
    virtual void mostrarInformacion() const {
        std::cout << "Marca: " << marca << " | Modelo: " << modelo
            << " | Velocidad: " << velocidadActual << " km/h";
    }
};
```

```

    }
    virtual void acelerar(double incremento) {
        velocidadActual += incremento;
        std::cout << " -> [" << marca << "] Acelerando a " << velocidadActual << " km/h." << std::endl;
    }
};

```

Módulo 2: Clase Derivada Auto

```

// =====
// CLASE DERIVADA 1: Auto (Hereda de Vehiculo)
// =====
class Auto : public Vehiculo {
private:
    int numeroPuertas;
    bool aireAcondicionadoActivo;
public:
    Auto(std::string m, std::string mod, int puertas)
        : Vehiculo(m, mod), numeroPuertas(puertas), aireAcondicionadoActivo(false) {}
    ~Auto() override {
        std::cout << " [Destructor Auto] Destruyendo partes del Auto " << marca << std::endl;
    }
    void encenderAire() {
        aireAcondicionadoActivo = true;
        std::cout << " -> [" << marca << " " << modelo << "] Aire acondicionado ENCENDIDO." << std::endl;
    }
    // Sobrescritura polimórfica (override)
    void mostrarInformacion() const override {
        Vehiculo::mostrarInformacion(); // Reutiliza código de la clase base
        std::cout << " | Puertas: " << numeroPuertas
            << " | A/C: " << (aireAcondicionadoActivo ? "SI" : "NO") << std::endl;
    }
    void acelerar(double incremento) override {
        velocidadActual += (incremento * 1.1); // Los autos aceleran con respuesta asistida
        std::cout << " -> [Auto " << marca << "] Aceleración asistida: " << velocidadActual << " km/h." << std::endl;
    }
};

```

Módulo 3: Clase Derivada Motocicleta

```

// =====
// CLASE DERIVADA 2: Motocicleta (Hereda de Vehiculo)
// =====
class Motocicleta : public Vehiculo {
private:
    int cilindradaCC;
    bool cascoPuesto;
public:
    Motocicleta(std::string m, std::string mod, int cilindrada)
        : Vehiculo(m, mod), cilindradaCC(cilindrada), cascoPuesto(false) {}
    ~Motocicleta() override {
        std::cout << " [Destructor Motocicleta] Destruyendo Motocicleta " << marca << std::endl;
    }
};

```

```

void colocarCasco() {
    cascoPuesto = true;
    std::cout << " -> [" << marca << " " << modelo << "] Casco abrochado de forma segura." << std::endl;
}

void mostrarInformacion() const override {
    Vehiculo::mostrarInformacion();
    std::cout << " | Cilindrada: " << cilindradaCC << " cc"
        << " | Casco: " << (cascoPuesto ? "PUESTO" : "NO PUESTO") << std::endl;
}

void acelerar(double incremento) override {
    if (!cascoPuesto) {
        std::cout << " -> [ALERTA " << marca << "] ¡Peligro! Debe ponerse el casco antes de acelerar." <<
std::endl;
        return;
    }
    velocidadActual += (incremento * 1.35); // Respuesta deportiva
    std::cout << " -> [Moto " << marca << "] Aceleración rápida: " << velocidadActual << " km/h." << std:
:endl;
}
};

```

Módulo 4: Programa Principal y Ejecución Polimórfica

```

// =====
// PROGRAMA PRINCIPAL CON DEMOSTRACIÓN POLIMÓRFICA
// =====
int main() {
    std::cout << "=== DEMOSTRACIÓN DE HERENCIA Y POLIMORFISMO (TEMA 2.1) ===" << std::endl;
    // Uso de un vector de punteros a la clase base (Vehiculo*)
    std::vector<Vehiculo*> flota;
    flota.push_back(new Auto("Toyota", "Corolla", 4));
    flota.push_back(new Motocicleta("Yamaha", "MT-07", 689));
    flota.push_back(new Auto("Ford", "Mustang", 2));
    std::cout << "\n1. --- ESTADO INICIAL DE LA FLOTA ---" << std::endl;
    for (const Vehiculo* v : flota) {
        v->mostrarInformacion(); // Invocación polimórfica (enlace dinámico)
    }
    std::cout << "\n2. --- OPERACIONES ESPECÍFICAS DE CADA TIPO ---" << std::endl;
    static_cast<Auto*>(flota[0])->encenderAire();
    static_cast<Motocicleta*>(flota[1])->colocarCasco();
    std::cout << "\n3. --- ACELERACIÓN POLIMÓRFICA EN BUCLE ---" << std::endl;
    for (Vehiculo* v : flota) {
        v->acelerar(50.0); // Cada tipo responde según su propia implementación
    }
    std::cout << "\n4. --- LIBERACIÓN POLIMÓRFICA DE MEMORIA ---" << std::endl;
    for (Vehiculo* v : flota) {
        delete v; // Invoca los destructores en cascada gracias a 'virtual ~Vehiculo()'
    }
    flota.clear();
    return 0;
}

```

3. Guión Pedagógico Minuto a Minuto (Sesión de 80 Minutos)

Tiempo	Fase de la Sesión	Actividad Docente / Estudiante	Concepto POO Integrado
00' - 15'	Encuadre Teórico	Explicación en pizarrón de la jerarquía Vehículo -> (Auto, Moto). Contrastar composición vs herencia ('Es un').	Relación Is-a & Modificador Protected
15' - 35'	Codificación Base	Los alumnos transcriben la clase Vehículo y crean las subclases Auto y Motocicleta delegando al constructor base.	Sintaxis de Herencia & Constructores
35' - 55'	Laboratorio Polimorfismo	Implementación del vector de punteros Vehículo*, llamadas con virtual/override y prueba de aceleración.	Enlace Dinámico & vtable
55' - 70'	La Trampa del Destructor	Eliminar 'virtual' del destructor base y observar la destrucción incompleta de Auto/Moto en consola. Luego corregir.	Destructores Virtuales & Memory Leaks
70' - 80'	Cierre y Evaluación	Resolución del cuestionario de control y revisión de reportes técnicos.	Síntesis y Evaluación

4. Cuestionario de Control y Rúbrica de Evaluación

Preguntas de Control para el Entregable del Alumno:

1. ¿Por qué los atributos marca, modelo y velocidadActual se declararon como protected en la clase Vehículo en lugar de private?

R: Para protegerlos de modificaciones externas directas (cumpliendo el encapsulamiento), pero permitiendo que las clases derivadas Auto y Motocicleta accedan y manipulen sus valores directamente sin necesidad de getters/setters adicionales.

2. ¿Qué ocurre en tiempo de ejecución si se omite la palabra reservada virtual en el destructor de la clase base (~Vehículo()) al hacer delete sobre un puntero de tipo Vehículo* que apunta a un objeto Auto?

R: Ocurre una liberación incompleta de memoria. Solo se ejecuta el destructor de Vehículo, omitiendo el destructor de Auto, lo que provoca fugas de memoria (memory leaks) de los atributos o recursos específicos de la subclase.

3. Explique cómo la tabla de funciones virtuales (vtable) permite que la llamada v->acelerar(50.0) ejecute comportamientos diferentes para el Auto y la Motocicleta en el bucle principal.

R: En tiempo de compilación, el compilador genera un puntero oculto (vpvr) en cada objeto que apunta a la vtable de su clase real. En tiempo de ejecución (enlace dinámico), C++ consulta la vtable para resolver la dirección exacta del método sobrescrito correspondiente al objeto concreto.

Criterio	Sobresaliente (10)	Satisfactorio (8)	Suficiente (6)	Insuficiente (5)
Jerarquía y Herencia (30%)	Diseña la jerarquía con modificadores protected y delegación explícita a constructores base sin errores.	Aplica herencia pero realiza la inicialización base dentro del cuerpo del constructor.	Define la herencia pero utiliza miembros públicos o privados inadecuados.	No logra implementar la derivación sintáctica en C++.
Polimorfismo y vtable (35%)	Implementa métodos virtuales, override y demuestra enlace dinámico mediante un vector de punteros base.	Implementa métodos virtuales pero presenta ligeras dudas en el uso de override o punteros base.	Aplica sobrecarga de métodos ordinarios en lugar de sobrescritura virtual.	No comprende ni implementa el enlace dinámico.

Gestión de Memoria (20%)	Declara el destructor virtual base y libera correctamente los recursos dinámicos sin fugas de memoria.	Usa destructores pero omite el calificador virtual en la clase base.	Libera memoria manualmente de forma incompleta.	Provoca colapsos en tiempo de ejecución o fugas masivas.
Cuestionario y Reporte (15%)	Responde las 3 preguntas con rigor técnico y justifica la mecánica interna de la vtable.	Responde correctamente pero con explicaciones poco detalladas.	Responde de forma ambigua o incompleta.	No entrega el cuestionario o sus respuestas carecen de sustento.

5. Referencias Bibliográficas (Formato APA)

- Ceballos Sierra, F. J. (2015). *Programación orientada a objetos con C++* (4.ª ed.). RA-MA Editorial.
- Finochietto, J. M. (2010). *Programación Orientada a Objetos en C++*. Universidad Nacional de Córdoba (UNC-CONICET).
- García Peñalvo, F. J., & Hernández Simón, J. A. (2005). *2 - Clases y objetos en C++*. Departamento de Informática y Automática, Universidad de Salamanca.
- Garrido Carrillo, A. (2016). *Prácticas con C++: metodología de la programación* (2.ª ed.). Editorial Universidad de Granada.
- González Pérez, A. (2025). *El gran libro de programación en C++* (1.ª ed.). Marcombo.
- Sánchez Asenjo, J. (2010). *Herencia y relaciones entre clases*. Fundamentos de Programación (Unidad 7). www.jorgesanchez.net.
- Google (2026). Gemini Notebook [Asistente de Inteligencia Artificial] <https://notebook.google.com/>